

Lecture 11: Legacy Bitcoin transaction, transaction malleability, SegWit, P2WPKH

1) What is a legacy Bitcoin transaction?

A legacy Bitcoin transaction is the older Bitcoin transaction format. In simple words, a Bitcoin transaction takes some old coins that already exist as **unspent outputs**, proves you are allowed to spend them, and then creates new outputs locked to new owners. In the lecture, the old style uses **ScriptSig** as the unlocking part and **ScriptPubKey** as the locking part. If unlocking data satisfies the locking condition, the spending is valid.

Easy idea

Think of an old transaction output like a locked box.

- **ScriptPubKey** = the lock on the box
- **ScriptSig** = the key plus proof that lets you open it

If the key fits, the old box is opened and the money is moved into new boxes.

Example

Suppose Alia received some BTC earlier. That earlier transaction created an output locked to **Alia's public key hash**. Now Alia wants to pay Babar.

So the new transaction does this:

- input: points to Alia's old output
- ScriptSig: gives Alia's signature + public key
- output 1: locks some BTC to Babar
- output 2: maybe locks change back to Alia

That is the basic spending story in Bitcoin.

2) Multiple inputs and outputs

A Bitcoin transaction can have:

- multiple inputs
- multiple outputs

Why? Because your money is not stored as one neat balance like a bank account. It is stored as separate UTXOs.

Example

Say you need to pay 0.8 BTC, but you own:

- 0.5 BTC in one old output
- 0.4 BTC in another old output

You can combine both as inputs:

- total input = 0.9 BTC
- payee gets = 0.8 BTC
- maybe 0.099 BTC comes back to you as change
- 0.001 BTC becomes fee

So Bitcoin is more like collecting notes from your wallet than deducting from one account balance.

3) What is transaction malleability?

This is one of the most important quiz concepts.

Transaction malleability means that the **transaction ID (txid)** of a transaction can be changed **without changing the actual payment meaning**. In the lecture, this is a problem because wallets track transactions by txid, and off-chain systems may depend on that txid staying fixed.

Easy meaning

Imagine you send a parcel and the parcel tracker number changes even though the parcel is still the same parcel.

Now everyone gets confused:

- wallet thinks, "Where did my transaction go?"
- network sees two versions
- systems built on top of it break

Why can this happen?

Because in legacy transactions, the signature data is part of the transaction data that is hashed to form the txid. If someone tweaks that signature representation in a valid way, the txid can change.

The payment is still basically the same, but the identifier changes.

4) Why is transaction malleability bad?

The slides give a few reasons, and these are very testable.

a) Unreliable transaction IDs

Wallets track sent transactions using txid. If txid changes, the wallet may think the transaction is different or missing.

b) Double-spend confusion

Attackers may alter a pending transaction's signature and rebroadcast a modified version. Even if the coins are not magically duplicated, this creates confusion about which version gets mined.

c) Network confusion

Different nodes may temporarily see different versions with different txids.

d) Breaks Layer-2 protocols

This is the biggest one conceptually. Payment channels and early Lightning-style systems depend on knowing exact txids in advance. If txid can change, the child transactions built on top of it may no longer point correctly.

Easy example

Suppose transaction A will later be spent by transaction B.

If B says:

"I spend output from txid = XYZ"

but later A's txid changes from XYZ to ABC because of malleability, then B is now referring to the wrong parent. So B breaks.

That is why transaction malleability was such a serious issue.

5) How does SegWit fix it?

SegWit means **Segregated Witness**. The idea is simple: separate the witness data, meaning the signature-related unlocking proof, from the part used to compute txid. In the slides, the witness proves authorization, but **txid is not created using ScriptSig** anymore.

Easy meaning

Before SegWit:

- signature data was mixed into the transaction ID

After SegWit:

- signature data is moved aside into the witness area
- txid is computed without that witness data

So changing witness data no longer changes txid in the same way.

Why this helps

Now the transaction has a stable identity. That makes Layer 2 and payment channels practical.

6) Legacy P2PKH vs SegWit P2WPKH

This comparison is very important.

Legacy P2PKH

P2PKH = Pay to Public Key Hash

In legacy style:

- the output locks funds to a public key hash
- input contains **ScriptSig = signature + public key**
- the input unlocking data is visible right there

SegWit P2WPKH

P2WPKH = Pay to Witness Public Key Hash

In SegWit:

- ScriptSig is empty
- unlocking data goes into the **witness**
- the output looks simpler on-chain
- txid is safer from malleability issues

One-line memory trick

- **legacy**: unlocking data in ScriptSig
 - **SegWit**: unlocking data in witness
-

7) “No op codes” in P2WPKH, then how does validation work?

This part confuses a lot of people, but it is actually simple once you see the idea.

The slide says that in P2WPKH, it looks like there are “no op codes” in the visible script, but Bitcoin internally expands it into a **virtual script**:

```
OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

Easy explanation

On-chain, SegWit output looks short and neat. But when validating, Bitcoin still knows the real logical checks it must do.

So internally it behaves like:

1. duplicate the public key
2. hash the public key
3. compare it with the stored public key hash
4. verify signature

So the authorization logic is still basically:

- does this public key match?
- does this signature verify?

Story version

Think of it like a zip file.

From outside, it looks small and simple.
But when Bitcoin validates it, it “unzips” the logic and executes the full meaningful checks.

8) P2WPKH execution idea

The lecture flow is roughly:

- Bitcoin sees SegWit output
- it expands it into virtual P2PKH-like logic
- witness items are pushed to stack
- op codes execute
- if checks succeed, spending is valid

You do not need to memorize every stack line unless you make you simulate it, but you should know the story:

What is on the witness?

Usually:

- signature
- witness public key

What gets checked?

- hash(public key) matches expected hash
- signature is valid for that transaction

If both are correct, the output is unlocked.

Lecture 12: Multisig and Merkle trees/proofs

9) What is multisig?

Multisig means **multiple signatures** are required to spend funds. The lecture defines it as an **M-of-N** setup. Example: **2-of-3 multisig** means any 2 valid signatures out of 3 allowed keys can authorize spending.

Easy meaning

Instead of one lock needing one key, imagine a safe that needs 2 out of 3 managers to approve opening it.

Example

Public keys belong to:

- Alice
- Bob
- Charlie

In a 2-of-3 multisig:

- Alice + Bob can spend
 - Bob + Charlie can spend
 - Alice + Charlie can spend
 - only Alice alone cannot spend
-

10) Why use multisig?

The slides mention several reasons.

a) Security

If one key is lost or stolen, funds may still be protected.

b) Shared accounts

A company treasury may require approval from more than one person.

c) Escrow

Buyer, seller, arbitrator in a 2-of-3 model.

d) Lightning Network

Payment channels use 2-of-2 multisig.

11) How does multisig work?

The lecture explains two parts.

Locking side

Bitcoin is sent to a script like:

```
OP_2 <PubKey1> <PubKey2> <PubKey3> OP_3 OP_CHECKMULTISIG
```

Meaning:

- 2 signatures required
- out of 3 listed public keys

This script is usually wrapped into an address form like P2SH or P2WSH.

Unlocking side

To spend it, the spender provides:

- the required number of valid signatures
- the witness script / redeem logic proving what the condition was

Then Bitcoin checks:

- do these signatures match enough of the allowed public keys?

If yes, spend succeeds.

12) P2WSH multisig execution

The lecture shows multisig execution under **P2WSH**. Main idea:

1. the last witness item must be the witness script
2. Bitcoin hashes that witness script
3. it compares the hash with the 32-byte value in the old output's scriptPubKey
4. if equal, continue
5. then witness items are pushed

6. witness script is executed
7. signatures are matched against public keys
8. if enough valid signatures exist, success

Easy story

Think of P2WSH like this:

- the old output does not store the whole long rule
- it stores only the hash of the rule
- when you spend, you must reveal the real rule
- Bitcoin checks: “Does this revealed rule hash to the expected value?”
- if yes, then it runs the revealed rule

This is like showing the original answer sheet and proving it matches the sealed fingerprint stored earlier.

13) Example of 2-of-3 verification

Suppose the public keys are A, B, C.

Required signatures = 2.

If spender provides:

- sigA
- sigC

Bitcoin checks:

- does sigA match one of A/B/C? yes, A
- does sigC match one of A/B/C? yes, C

Count valid = 2, so success. That is exactly the logic shown in the execution slides.

Important concept

Bitcoin does not need all 3 signatures in a 2-of-3 scheme. It only needs 2 valid ones.

14) What is a Merkle tree?

A Merkle tree is a hash tree used to summarize all transactions in a block into one final hash called the **Merkle root**. That Merkle root is stored in the block header.

Easy example

Suppose a block has 4 transactions:

- Tx1
- Tx2
- Tx3
- Tx4

First hash them individually:

- $h1 = \text{Hash}(\text{Tx1})$
- $h2 = \text{Hash}(\text{Tx2})$
- $h3 = \text{Hash}(\text{Tx3})$
- $h4 = \text{Hash}(\text{Tx4})$

Then combine pairwise:

- $h12 = \text{Hash}(h1, h2)$
- $h34 = \text{Hash}(h3, h4)$

Then combine again:

- $\text{root} = \text{Hash}(h12, h34)$

That final root summarizes the whole set.

Why useful?

If even one transaction changes, its leaf hash changes, then upper hashes change, and finally the Merkle root changes too.

So it is great for tamper detection.

15) How do you verify that a transaction is in a block?

The lecture gives 3 ideas:

1. **Check inclusion**
confirm the transaction is part of the block

2. **Validate block integrity**
ensure the transaction's hash contributes correctly to the block's Merkle root
3. **Confirm block validity**
ensure the block itself is valid and part of the accepted chain

Easy example with Merkle proof

Suppose you want to prove Tx2 is in the block.

You don't need the whole block.

You only need the sibling hashes along the path.

If tree is:

- h1, h2, h3, h4
- $h_{12} = \text{Hash}(h_1, h_2)$
- $h_{34} = \text{Hash}(h_3, h_4)$
- $\text{root} = \text{Hash}(h_{12}, h_{34})$

To prove Tx2, you need:

- h1
- h34

Then verifier does:

- compute $h_2 = \text{Hash}(\text{Tx2})$
- compute $h_{12} = \text{Hash}(h_1, h_2)$
- compute $\text{root} = \text{Hash}(h_{12}, h_{34})$

If this matches block header's Merkle root, Tx2 is proven to be included.

16) What is a Merkle proof?

A Merkle proof is the small set of sibling hashes needed to prove a transaction belongs to a block. The lecture says this allows lightweight verification and needs only about $\log_2(N)$ hashes instead of the whole block.

Why this is amazing

If a block has thousands of transactions, you still do not need to download all of them just to verify one transaction.

That is efficient and scalable.

Exam-style line

Merkle proof proves **membership** of a transaction in a block without revealing or downloading the entire block.

17) Why Merkle tree matters

The slides highlight these benefits:

- summarizes all transactions
- enables lightweight verification
- detects tampering
- improves scalability

One-line memory trick

Many transactions in, one root out.

Lecture 13: Block header, Bitcoin micropayment problem, scripts revisited

18) What is a Bitcoin block header?

The block header is the short summary section of a block. The lecture says it has **6 fields**:

1. Version
2. Previous block hash
3. Merkle root hash
4. Timestamp
5. Target/Difficulty
6. Nonce

And the block header hash is:

`SHA256(SHA256(Block_Header))`

19) Easy meaning of each field

1. Version

Tells which protocol rules/signaling version is being used.

2. Previous block hash

Links this block to its parent. This is what creates the chain.

3. Merkle root hash

Summarizes all transactions in this block.

4. Timestamp

Approximate block creation time.

5. Target / difficulty

Sets how hard mining is. It adjusts to keep average block generation near 10 minutes.

6. Nonce

A changing number miners keep trying to get a valid block hash.

Easy analogy

Think of a block header like a book cover summary:

- which edition
- previous chapter reference
- summary of all contents
- time
- difficulty rule
- guess number used in mining

20) Why can't Bitcoin directly work well for coffee payments?

This is the main motivation section in Lecture 13. The lecture asks: can we pay for a coffee with BTC directly? And it points out some problems:

- average block time is around 10 minutes
- Bitcoin handles only about 7 transactions per second
- it is not ideal to broadcast every tiny payment to the whole world
- fees can be too large relative to a tiny purchase

Easy meaning

Buying coffee needs:

- fast confirmation
- tiny fee
- high volume support

Bitcoin base layer is secure, but not designed for every tiny daily payment directly on-chain.

Example

If your coffee costs around 900 sats, and the fee is also a few hundred sats, that is a big fraction of the payment. That feels inefficient.

21) What can Bitcoin script do?

Lecture 13 revisits opcodes and explains what scripts can express.

Bitcoin Script can:

- check signatures with `OP_CHECKSIG`
- combine conditions with `OP_IF`, `OP_ELSE`, `OP_ENDIF`
- require multiple approvals with `OP_CHECKMULTISIG`
- lock by time with `OP_CHECKLOCKTIMEVERIFY`
- lock by delay with `OP_CHECKSEQUENCEVERIFY`

Easy meaning

Bitcoin Script is like a rule engine for spending money.

It can say things like:

- Alice must sign

- or Bob can sign after a date
- or 2 of these 3 people must approve
- or secret value S must be revealed

So Bitcoin is not just “send coin”. It can encode spending rules too.

22) Important spending types you should know from this lecture

The slides list several forms.

- P2PKH
- P2MS
- P2SH
- P2WPKH
- P2WSH
- P2TR
- CLTV / CSV
- hash locks
- HTLC

For your quiz, the main thing is not full memorization of every deep implementation, but knowing the role:

- **P2PKH**: pay to public key hash
 - **P2WPKH**: SegWit version of single-key spend
 - **P2WSH**: SegWit version where full script hash is used
 - **P2TR**: Taproot output type
 - **CLTV/CSV**: time-based locks
 - **hash lock / HTLC**: spending depends on revealing a secret hash preimage
-

23) What is Miniscript?

Lecture 13 gives a nice analogy: Miniscript is like **TypeScript for Bitcoin**. It is more structured and readable, and turns into raw Bitcoin Script.

Easy meaning

Raw Bitcoin Script is low-level and easy to mess up.
Miniscript is a more human-friendly way to describe spending policies safely.

Why useful?

Because real scripts get complex quickly, and developers need a clearer, safer abstraction.

24) Why this lecture starts moving toward Layer 2

Once the lecture says:

- Bitcoin is slow for tiny payments
- fees depend on size
- we need secure off-chain ways

it is basically preparing your brain for payment channels and Lightning later.

Even though Lecture 14 is excluded, Lecture 13 already sets the reason:
base layer is secure but not ideal for everyday micropayment traffic.

Very important “understand this like a story” summary

Here is the whole thing as one simple story:

Bitcoin transactions originally used the legacy format, where unlocking data sat inside the transaction in a way that could affect txid. That created the transaction malleability problem. This was bad especially for systems that needed stable txids, such as Layer 2 ideas. SegWit fixed this by moving witness data away from the txid calculation. In SegWit single-key transactions, we use P2WPKH, where the witness contains the signature and public key, and Bitcoin internally reconstructs the necessary logic to verify ownership. Then the lecture moves to multisig, where spending can require M out of N signatures, which is useful for shared control, escrow, and Lightning-style channels. After that, the lecture explains Merkle trees, which compress all transactions of a block into one root in the block header, making transaction inclusion proofs efficient. Finally, Lecture 13 explains the block header fields and asks a practical question: can Bitcoin base layer really handle everyday coffee payments? Since on-chain transactions are comparatively slow and not ideal for massive tiny payments, the course begins motivating more scalable payment approaches.

Quick exam-style definitions

Legacy transaction

Old Bitcoin transaction format where unlocking data is carried in ScriptSig.

ScriptPubKey

The locking script placed in an output.

ScriptSig

The unlocking data used in legacy inputs.

SegWit

Segregated Witness, which moves witness/signature data out of the txid-critical part of the transaction.

Transaction malleability

Ability to change a transaction's txid without changing its real payment meaning.

P2WPKH

SegWit output type for paying to a witness public key hash.

Multisig

A transaction requiring M valid signatures out of N allowed keys.

P2WSH

SegWit output type that stores the hash of a witness script.

Merkle tree

A hash tree that summarizes all transactions in a block.

Merkle proof

The sibling hashes needed to prove a transaction belongs to a block.

Block header

Compact summary of block metadata used for linking, mining, and validating the block.

What sir can ask from these lectures

Very likely questions:

- Explain transaction malleability and why it is a problem
- How does SegWit solve transaction malleability?
- Difference between legacy P2PKH and SegWit P2WPKH
- In P2WPKH, why do slides say “no op codes” but validation still works?
- What is multisig? Explain 2-of-3
- How does P2WSH multisig validation work?
- What is a Merkle tree?
- What is a Merkle proof?
- How can you verify that a transaction belongs to a block?
- What are the six fields of the block header?
- Why is Bitcoin not ideal for coffee payments directly on-chain?
- What can Bitcoin opcodes/scripts do?